天链机械臂系统·小白快速上手手册（V8）

这套文档给谁看：第一次接触这套机械臂、需要在 Nano 板子上操作的人。

读完你能做到：会远程连上 Nano → 会开机 → 会看版本档案选版本 → 会示教 → **会直接发命令看到机械臂干活** → 会查"已经示教了哪些点" → 会传程序、更新程序 → 二次开发知道每个程序是干什么的、改哪里。

适用系统：Jetson 板（Nano）× TCB605 机械臂 × D435 相机 × ROH 灵巧手 × MQTT

更新日期：2026-10-08

目 录

0. 30 秒看懂这套系统 5

1. 先认识"编号"（看懂命令里的 A1-05） 6

2. 名词小词典（小白版） 7

3. 怎样远程进入 Nano 8

方式 A：图形远程桌面（NoMachine）—— 日常推荐 8

方式 B：SSH 命令行 —— 只敲命令时用（快、省资源） 8

Nano 的 IP 从哪看？ 8

连不上排查 8

4. 开机准备（每次必做） 10

5. 版本档案：程序一共有几版、都在哪、怎么选 11

5.1 功能演进时间线（按实际操作的记录整理） 11

5.2 怎么选（推荐） 12

5.3 怎么分辨"我正在跑的是哪版" 12

6. 四大主程序（先认门） 13

7. 程序一【种子示教】怎么用 15

8. 程序二【叉子示教】怎么用 16

第 0 步（一次性，做一遍就行） 16

教叉子下排盆（A1/A4/B1/B4）的小流程 16
9. 程序三【种子运行】怎么用 18
10. 程序四【叉子运行】怎么用 19
11. 已经示教了哪些点（截至 2026-10-08，以第 11.3 节工具实际刷新为准） 20
11.1 种子（basket_aruco.yaml）——已完整示教 14 盆，全部是上排 20
11.2 叉子（fork_basket_teach.yaml）——已示教 1 盆 + 3 个全局点 20
11.3 怎么随时看最新、最准的清单（新增小工具，推荐每次用前跑一遍） 21
12. 现在就能运行的命令（复制即用） 23
12.1 种子：下面 14 个盆都能跑（A3 排用新版，其余老版新版都行） 23
12.2 叉子：30 号（A2-06）能跑全套（用新版） 24
12.3 急停（备着） 24
12.4 以后教了新盆，怎么自己拼命令 24
13. 程序里最该认识的几个数字（关键参数在哪） 25
14. 怎么把 MQTT 命令发出去（两种方式） 26
15. 怎样下载/上传程序、怎样更新程序 27
15.1 把程序从电脑传到 Nano（上传） 27
15.2 传完之后必做两件事 27
15.3 怎样更新程序（三种场景） 27
16. 数据文件都存在哪（别乱删！） 29
17. 二次开发（一）：所有程序逐一详解 30
17.1 程序地图（谁启动谁、谁继承谁） 30
17.2 种子示教程序 `tl_hand_deploy/aruco_basket_teach.py` 30
17.3 种子运行程序 `tl_hand/mock_pick_place.py` 31
17.4 叉子示教程序 `tl_fork_teach/fork_basket_teach.py` 31
17.5 叉子搬运核心 `tl_fork_teach/fork_basket_cycle.py` 32
17.6 叉子运行入口 `tl_fork_teach/fork_trim_mqtt.py` 33
17.7 MQTT 协议模块 `tl_hand/mqtt_protocol.py` 33
17.8 启动脚本（4 个 .sh）一文看懂 34
17.9 查点小工具 `list_taught.py`（V3 新增） 34
17.10 其它程序一览（标定 / 测试 / 历史，一般不用碰） 34
18. 二次开发（二）：改码流程与调试 36

18.1 先记住三条 36
18.2 "我想改 X → 改哪个文件" 36
18.3 改文件 ↔ 哪个脚本生效 ↔ 要不要编译（总表） 36
18.4 标准开发流程（5 步） 38
18.5 调试常用（在 Nano 上） 38
18.6 强烈推荐：VS Code 远程开发 38
18.7 红线 38
19. 小白常见问题速查表 39
20. 最快上手路线（照这个顺序做就行） 41

（首次打开：右键目录 → 更新域 → 更新整个目录）

修订	日期	内容
初版	2026-10-07	系统总览、编号规则、示教/运行操作、MQTT 命令、FAQ
V2	2026-10-07	老版/新版双版本说明 + 选择建议；新增：远程进 Nano、程序传输、更新程序、二次开发入门
V3	2026-10-07	① **已示教清单**（种子 14 盆 / 叉子 30 号 + 全局点）与**现成可跑命令**逐条列出（第 11、12 章）② **版本档案**（三套程序 + 功能时间线，第 5 章）③ **二次开发扩写为逐程序详解**（第 17 章：所有程序一张一张介绍）④ 新增查点小工具 `list_taught`
V4	2026-10-07	① **叉子待机位统一为种球默认位**（`run_fork_mqtt.sh` 参数；启动/作业结束都回这里）② **下排观测位 B 侧支持自动 = A 侧 +180°** 推导（叉子/种子运行程序；手动记录优先）
V5	2026-10-07	**叉子 `trim_pick` 检测失败后自动回待机位**（对齐种子流程）：一直检测不到码时不再干停在识别位，失败报 `retCode 3` 后自动回待机位，排查原因后重发同一条命令即可重试
V6	2026-10-08	**下排盆（A1/A4/B1/B4）的 W/T 改为独立槽位**：教下排盆时按

		W/T 自动存成"下排专用"（上排那对不受影响）；运行下排盆自动用下排那对（未教前沿用上排并打印 WARNING）。pick 与返回路径都按盆的排选对
V7	2026-10-08	**A3 排 ArUco 码贴反的软件修正**：4 个脚本内置 `--reversed-rows A3`——输入的位置号自动镜像成该格实际贴的码（A3-01↔ID71 … A3-24↔ID48）；已教 A3 盆的新命令、补教方法见第 11.1 / 19 章
V8	2026-10-08	① 第 1 节补 **ID ↔ 编号换算**（ID34 → 编号 35 → A2-11）② 第 8 节新增 **`O / v / g` 三个"位"对照表**，FAQ 新增"按 O 去的是种球默认位（不是看花篮）"一条

📖 配套文档：桌面《TL_lowerobs_test_操作说明》是"新版功能（下排观测位 / 修剪新点位）"的详细验证说明；本手册是"从头上手"的极简版。

0. 30 秒看懂这套系统

硬件（一句话版）：

部件	干什么的
机械臂（6 个关节 J1~J6）	干活的"手"，能抓种子、叉花篮
腕上相机（D435）	机械臂的"眼睛"，用来认盆上贴的黑白方块码（ArUco 码）
灵巧手 / 叉子	末端工具：种子流程用灵巧手，修剪流程用 U 形叉
Nano 小电脑	"大脑"，上面跑所有程序
小车 / 上位机	通过 **MQTT** 给机械臂发命令（比如"给 A1-05 放种子"）

软件怎么协作：

```
小车发命令 ──MQTT──▶ Nano 上的"运行程序" ──▶ 驱动机械臂做动作
                          ▲                          │
                          └── MQTT 反馈结果 ◀────────┘

"示教程序"是另一个角色：你手动拖着机械臂走一遍，它把位置记到 YAML 文件；
"运行程序"干活时就照着这份 YAML 走。
```

两套作业：

- **种子作业**：从固定点抓一粒种子 → 走到花盆处识别码 → 放进盆里 → 回待机位。
- **修剪作业**：把花篮用叉子叉起来 → 搬到修剪位（等小车剪根）→ 再放回原处。

1. 先认识"编号"（看懂命令里的 A1-05）

- 整个架子共 **192 个盆** = 8 排 × 每排 24 个；
- 排的名字：**A1 A2 A3 A4 B1 B2 B3 B4**；盆的总编号 1~192。

排	盆号范围	排	盆号范围
A1	1 ~ 24	B1	97 ~ 120
A2	25 ~ 48	B2	121 ~ 144
A3	49 ~ 72	B3	145 ~ 168
A4	73 ~ 96	B4	169 ~ 192

- 命令里用 **"排-序号"** 的写法：`A1-05` = A1 排第 5 个 = 总编号第 5 号盆。

  换算公式：排 =（盆号−1）÷24；A2-06 = 24+6 = 盆号 30。

- **ArUco 码 ID ↔ 编号：编号 = ID + 1**（ID 0..191 ↔ 编号 1..192）。

例：看到 **ID34** → 编号 35 → 减 1 除以 24 得排（A2）、余数+1 得第几格（第 11 格）→ **A2-11**。（A3 排的码是倒贴的，属特例、已由程序自动换算，见第 11.1 节。）

- **上排 / 下排**：A1、A4、B1、B4 这四排位置低，叫**下排**（新版程序会自动先去做一个"观测位"步骤）；A2、A3、B2、B3 是**上排**。

2. 名词小词典（小白版）

名词	人话解释
示教	你用手拖着机械臂走到某个位置、按下按键把位置"记下来"。机械臂不会自己知道盆在哪，必须先教
运行	程序照着教好的位置自动干活
检索位（search pose）	某盆干活时的"站位"：能看到这盆的码、能伸过去操作（按 B 时记录）
落种接近位(A) / 落种释放位(P)	种子流程里"手先靠近盆口上方" / "真正把种子放进盆里"的两个位置（教一次每盆）
下排观测位	下排盆专用的"看码位置"，分 A / B 两侧各记一个（只有新版有）
待机（观测）位	没活干时站着待命的位置（启动、作业结束回到这里；2026-10-07 起统一为种球默认位 `-23.246, -14.220, 53.738, 4.389, 26.746, -3.532`）；**不用于识别码**，别和下排观测位混淆
W 载运点 / T 暂放位	叉子搬运花篮路上的两个中转点：**上排用一对（教一次）；下排（A1/A4/B1/B4）用另一对**（2026-10-08 起，教下排盆时按 W/T 自动存下排专用）
修剪位 1 / 2	两个**写死在程序里**的固定位置，不用教（花篮最终放在那里等剪根）
ArUco 码	盆上贴的黑白方块码，相机认它来识别盆
ServoJ	机械臂平滑运动的执行方式；日志里 `streaming / ServoJ points` 都是它在工作
MQTT	消息通道：命令走 `node/arm/01/down`，反馈走 `node/arm/01/up`
retCode	反馈里的结果码：**0**=成功、**1**=已接受/开始、**2**=位置被占/忙、**3**=失败（含识别不到）
targetId	命令里要操作的盆，如 `"A1-05"`

3. 怎样远程进入 Nano

Nano 就是一台带 Ubuntu 的小电脑。你有两种方式"进去"：

方式 A：图形远程桌面（NoMachine）—— 日常推荐

- 在**电脑**上打开 NoMachine（桌面有图标）；
- 新建连接（New connection），填 **Nano 的 IP 地址**（以前连过的话，历史列表里就有）；
- 输入 Nano 的用户名（`nvidia`）和密码；
- 进入 Ubuntu 图形桌面后，在桌面**右键 → Open Terminal** 打开终端，就能敲本文档里的所有命令。

方式 B：SSH 命令行 —— 只敲命令时用（快、省资源）

在 Windows 的"终端 / PowerShell / Git Bash"里输入：

```
ssh nvidia@<Nano 的 IP>
# 例如：ssh nvidia@192.168.1.50
```

- 第一次连接会问 `yes/no` → 输 `yes`；
- 然后输密码（**输入时屏幕不显示任何字符，是正常的**）；
- 看到提示符变成 `nvidia@ubuntu:~$` 就成功了。

Nano 的 IP 从哪看？

- 能在 Nano 上操作时，打开终端执行 `hostname -I`，输出的第一串就是（形如 `192.168.x.x`）；
- 或者看路由器后台的"已连接设备"列表；
- ⚠ 换网络/重启路由器后 IP 可能变化，连不上先查 IP。

连不上排查

现象	先查什么
一直连不上	电脑和 Nano 是不是同一个网络（同一个 WiFi/交换机）？
之前能连现在不行	Nano 的 IP 变了（重新 `hostname -I`）？Nano 关机了？
SSH 提示密码错误	大小写/输入法（密码里的符号要在英文输入法下敲）

4. 开机准备（每次必做）

前提：已经能进入 Nano（第 3 节）。

```
# 终端 A：先启动机械臂驱动（约 30 秒；看到它开始持续刷状态才算就绪）
ros2 launch tl_driver tl_tcb605_driver.launch.py

# 终端 B：给脚本加执行权限（从 Windows 拷贝到 Nano 会丢这个权限，做一次即可）
chmod +x ~/Desktop/TL_lowerobs_test/scripts/*.sh
chmod +x ~/Desktop/TL/scripts/*.sh
```

四条纪律（务必遵守）：

- **物理急停在手边**——程序里的一切"停止"都不等于断电；
- **同一时间只开一个会发运动的程序**（示教/运行不能同时开）；
- **动作没走完不要按 Ctrl+C**——会让机械臂停在半路（见第 19 节速查）；
- **老版和新版程序不要同时跑**（会互相抢控制权）。

5. 版本档案：程序一共有几版、都在哪、怎么选

Nano 桌面上存在过这几套程序，都以 `TL` 开头：

文件夹	俗称	状态	有什么 / 没什么
`~/Desktop/TL`	**稳定版（老版）**	✅ 日常生产用	种子（两阶段 MQTT 反馈）+ 叉子（W/T 示教 + 修剪位 1/2 + 作业状态文件）；**没有**下排观测位
`~/Desktop/TL_lowerobs_test`	**验证版（新版）**	🧪 当前测试用	稳定版全部功能 + **下排观测位**（v/d/g、A/B 分侧、运行端自动路由）；修剪点位与稳定版相同（都是新读数）
`~/Desktop/TL_Handback`	早期工作区（存档）	⛔ 不要使用	更早的一版程序，里面还有很旧的编译产物（build/install）；只当备份留着，别拿它跑

5.1 功能演进时间线（按实际操作的记录整理）

时间	发生了什么
早期（8 月及以前）	最初版本：种子 + 叉子基础流程（对应 `TL_Handback` 存档；它的 build/install 是那时编译的）
之后	稳定版 `~/Desktop/TL` 陆续增加：叉子修剪流程（示教 W/T + 写死的修剪位 1/2）、种子两阶段反馈（seed:1→0→seed_place:1→0）、叉子"作业状态文件"（防没放回就再抓）
2026-10-06	开发**下排观测位**：先试过"固定参数"方案，最终改为**现场拖拽记录**（v 切侧 / d 记录 / g 验证 / A/B 分侧）→ 抄出完整程序生成独立验证包 `TL_lowerobs_test`
2026-10-07	**修剪位 1/2 换成新的示教器读数**（`TL` 和 `TL_lowerobs_test` 两边源码都已更新）；叉子"作业状态文件"残留问题处理；**叉子待机位统一为种球默认位；下排观测位 B 侧改为自动按 A 侧 +180° 推导**（验证包先行）

会 git 的话：主程序目录保留了提交历史（修剪流程、两阶段反馈、下排观测位几次改动都能查到）。

5.2 怎么选（推荐）

任务	用哪个版本	为什么
种子·上排盆（A2/A3/B2/B3 放种）	⭐ **老版** `~/Desktop/TL`	成熟稳定；上排本来就不需要观测位
种子·下排盆（A1/A4/B1/B4 放种）	**新版** `TL_lowerobs_test`	老版没有"下排观测位"，下排识别不了
叉子（上排/下排）	**新版**（建议统一）	新版带下排观测位；两版修剪点位一致

三条注意：

- 两版**共用同一份示教数据**（YAML 文件），教出来的数据两边通用；
- 两版**绝对不能同时运行**；
- 怎么区分你在开哪版：**看你敲的命令**里写的是 `TL` 还是 `TL_lowerobs_test`——就这么简单。

5.3 怎么分辨"我正在跑的是哪版"

- 看命令路径：有 `TL_lowerobs_test` = 新版；只有 `TL` = 老版；
- 看启动打印：**新版示教程序**会打印 `Current side: A …` 和 `New keys: v switch side | d record lower-observation pose | g goto it.`；老版没有这两行；
- 看运行日志：新版叉子运行下排盆时会打印 `localizing marker at the lower-row observation pose...`。

6. 四大主程序（先认门）

程序	作用	老版命令	新版命令
种子示教	教"每个盆放种子的位置"	`bash ~/Desktop/TL/scripts/teach_seed.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/teach_seed.sh`
叉子示教	教"每个盆叉花篮的位置"+ 全局点	`bash ~/Desktop/TL/scripts/teach_fork.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/teach_fork.sh`
种子运行	等 MQTT 命令，自动抓种子放入盆	`bash ~/Desktop/TL/scripts/run_seed_mqtt.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/run_seed_mqtt.sh`
叉子运行	等 MQTT 命令，自动叉花篮去修剪/放回	`bash ~/Desktop/TL/scripts/run_fork_mqtt.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/run_fork_mqtt.sh`
查点小工具（新增）	列出所有已示教的盆和点位（只读，不动机械	`bash ~/Desktop/TL/scripts/…`（老版没有，见第 11.3 节）	`bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh`

	臂）		

脚本里到底跑了什么（了解原理，看日志不迷糊）：

```
teach_seed.sh   → tl_hand_deploy.aruco_basket_teach     （种子示教程序）
teach_fork.sh   → tl_fork_teach.fork_basket_teach       （叉子示教程序）
run_seed_mqtt.sh → tl_hand.mock_pick_place              （种子运行程序）
run_fork_mqtt.sh → tl_fork_teach.fork_trim_mqtt         （叉子运行程序，核心逻辑在
fork_basket_cycle.py）
list_taught.sh  → list_taught.py                        （查点小工具）
```

脚本做的事：加载 ROS 环境 → 把源码目录塞进 `PYTHONPATH` → 用 `python3 -m` 启动程序。**所以修改源码 `.py` 文件后，重启脚本就生效，不需要编译。**

7. 程序一【种子示教】怎么用

干什么：把"每个盆的放种位置"教给机械臂，存进 `~/tl_mock_pick_place/basket_aruco.yaml`。

```
# 只教上排盆 → 推荐老版（⚠️ A3 排例外：A3 码贴反，教它要用新版，见第 11.1 节）：
bash ~/Desktop/TL/scripts/teach_seed.sh
# 要教下排盆（要先记录观测位）→ 必须用新版：
bash ~/Desktop/TL_lowerobs_test/scripts/teach_seed.sh
```

启动后：机械臂自动去 A 侧起始位；电脑上弹出相机画面（能看到识别到的码）。

按键人话解释：

按键	意思	会自己动吗	两版差异
数字键 / `c`	打盆编号 / 清空重输	不动	相同
`B`	**选定这盆**：锁定当前看到的码 + 记住现在站的位置（检索位）	不动	相同
`A`	记"落种接近位"（手先靠近盆口上方）	不动	相同
`P`	记"落种释放位"（手到达放种子的位置）	不动	相同
`F / I / L / T`	叉子相关的备用记录键	不动	相同
`R`	回到当前盆的检索位（一次自动运动，等它走完）	✅ 会	相同
`Q`	退出程序	不动	相同
`v / d / g`	切侧 / 记录下排观测位 / 回位验证	v、g 会动	**只有新版有**（老版没有"记录下排观测位"的功能）

标准操作（每盆 5 步）：

```
① 按住臂上的拖拽钮，把臂拖到"能看清目标盆的码"的位置
② 输编号 → 按 B      （⚠ 手先松、臂停稳 1~2 秒再按，否则报"看不清/样本不稳"）
③ 拖到落种接近位 → 按 A
④ 拖到落种释放位 → 按 P
⑤ 按 R 回到检索位 → 拖向下一盆，重复 ①~⑤
```

下排盆（A1/A4/B1/B4）额外前置（用新版）：先各教一次两侧观测位——

```
拖到能看清 A 区下排码处 → 停稳 → 按 d（打印 side=A 就对）→ 按 g 验证能回位
按 v 切到 B 侧 → 拖到位 → 停稳 → 按 d → 按 g 验证
```

每次按完 `d`，**看一眼打印的 side=A 还是 B**，别记反。

8. 程序二【叉子示教】怎么用

干什么：教"每个盆的叉取位置（F/I/L）"+ 两个全局点（W/T）+ 待机位（O）+ 下排观测位，存进 `~/tl_fork_teach/fork_basket_teach.yaml`。

```
# 建议统一用新版：
bash ~/Desktop/TL_lowerobs_test/scripts/teach_fork.sh
# 老版（没有 v/d/g 下排观测位那组键，其余操作相同）：
bash ~/Desktop/TL/scripts/teach_fork.sh
```

启动后：**不动臂**；只有 `v / g / R` 会自己动，其它全靠手拖。

第 0 步（一次性，做一遍就行）

- 按 `O` → 臂去**待机位**（2026-10-07 起为种球默认位 `[-23.246, …]`）并记录（这就是运行端"启动/干完活回来"的位置）；
- 下排观测位（要跑下排盆之前必须做）：确认当前侧是 A（按 `v` 切换，看打印 `Current side: A`）→ 拖到能看清 A 区下排码处 → 停稳 → 按 `d` → 按 `g` 验证；**B 侧不必记录**（运行端自动按 A 侧 +180° 推导，手动记录优先）。

教叉子下排盆（A1/A4/B1/B4）的小流程

```
① （第一次教下排前）确认第 0 步已完成（待机位 + A 侧观测位）
② 瞄准：按 g 到下排观测位附近 → 按住拖拽钮微调到"能看清这盆的码"的位置
③ 输编号（A1=1~24，A4=73~96，B1=97~120，B4=169~192）→ 停稳 1~2 秒按 B
   （按 B 的位置会记成该盆检索位——尽量选在盆正前方、方便伸叉的位置）
④ 拖到叉接近位按 F → 水平插入按 I → 竖直抬起按 L
   ＊ 第一盆下排要接着教 W/T：拖到"下排专用高位载运点"按 W → 拖到"下排专用暂放位"按 T
     （当前选中的是下排盆 → 自动存成"下排专用"槽位，上排原有的不受影响；2026-10-08 起）
⑤ 按 R 回检索位（等 "Returned: max joint error=…"）→ 换下一盆重复 ②~⑤
   （第 2 盆起：输编号 → B → F → I → L → R，W/T 不用再按）
```

B 区盆小技巧：教 B 区之前按 `v` 切到 B 侧起始位，再往前拖到目标盆；教完想回 A 侧再按一次 `v`。

运行时（trim_pick）：A 区下排去"下排观测位 A"识别（日志 `YAML lower_observation_pose_a.`）；B 区下排去自动推导的 B 侧位（日志 `derived … rotated 180 deg`）。

按键人话解释：

按键	意思
数字 / `c` / `B`	输编号 / 清空 / 选定这盆（记住站位）
`F`	记"叉子接近位"（叉尖对准盆口前方）
`I`	记"水平插入位"（叉子插进盆沿下方；会检查你是不是水平插的）
`L`	记"竖直抬起位"（叉子把盆微微抬离；会检查是不是竖直抬的）
`W`	记"高位载运点"（举着花篮走路的高点）——**全局教一次**；若当前选中的是**下排盆**，会自动存成"下

	排专用"
T	记"暂放位"（搬运链上花篮的中转点）——**全局教一次**；下排盆同理存成"**下排专用**"
O	去**待机位**并记录（已统一为种球默认位 `[-23.246, …]`；运行程序启动/干完活都回这里，2026-10-07 起）
v / d / g	切侧 / 记录下排观测位 / 回位验证（**只有新版有**）
R	回当前盆检索位（等看到 `Returned: max joint error=…` 才算走完）
Q	退出

三个"位"别搞混（按 O 去的是种球默认位，是正常设计）：

键	去的位置	干什么用
O	**待机位** = 种球默认位 `[-23.246, -14.220, 53.738, 4.389, 26.746, -3.532]`	启动/收工回位、刷新待机记录；**不是"看花篮"的位置**（2026-10-07 起统一）
v	**A / B 侧起始位** `-83.668… / 96.332…`（J1±180°）	教花篮的起点站位——**这就是老的"观测位"**；注意当前侧是 A 时第一下 v 会先去 B 侧，再按一下才回 A 侧
g	**下排观测位**（当前侧；d 记录）	看下排盆（A1/A4/B1/B4）的码；上排盆不看这里，直接去该盆检索位

标准操作：

```
第一盆（必须一次教完 W 和 T；**下排盆的第一盆同理**—那时按 W/T 会自动存成下排专用那对）：
① 输编号 → B → 拖到叉接近位按 F → 拖到水平插入按 I → 拖到竖直抬起按 L
② 拖到高位载运点按 W → 拖到暂放位按 T → 按 R 回检索位
以后每盆：
输编号 → B → F → I → L → R        （W/T 不用再按；重按会按"当前盆的排"覆盖上排或下排对应那一对，别误按！）
```

被拒绝不慌：按 I / L 时报 "horizontal travel is only…" 之类，是程序发现你拖得不够直/不够远，重新拖到位再按即可（阈值：水平 ≥20mm、竖直 ≥30mm、漂移 ≤60mm、姿态变化 ≤25°）。

9. 程序三【种子运行】怎么用

```
# 上排盆 → 推荐老版：
bash ~/Desktop/TL/scripts/run_seed_mqtt.sh
# 下排盆 → 必须用新版：
bash ~/Desktop/TL_lowerobs_test/scripts/run_seed_mqtt.sh
```

启动后它待机等命令。收到命令后自动执行：

```
固定点抓一粒种子 → （下排盆先去观测位）识别盆的码 → 对准 → 放入盆里 → 回待机位
```

反馈怎么读（朝上行的回执，一共 4 条，分两个阶段）：

```
{"cmd":"seed",       "targetId":"A1-05","retCode":1}   已接受，开始干活
{"cmd":"seed",       "targetId":"A1-05","retCode":0}   种子已放入
{"cmd":"seed_place", "targetId":"A1-05","retCode":1}   收尾（回位）
{"cmd":"seed_place", "targetId":"A1-05","retCode":0}   ✅ 全部完成
```

失败时：`retCode:3`（识别不成功/出错，`message` 里有原因）或 `retCode:2`（位置被占 / 程序正忙）。

10. 程序四【叉子运行】怎么用

```
# 建议统一用新版：
bash ~/Desktop/TL_lowerobs_test/scripts/run_fork_mqtt.sh
```

启动后（没有未完搬运时）先回到**待机位**（2026-10-07 起与种子程序统一：`-23.246, -14.220, 53.738, 4.389, 26.746, -3.532`），然后等命令：

命令	效果
`{"cmd":"trim_pick","targetId":"A2-06"}`	叉起花篮 → 搬到**修剪位 2** 停住（等小车剪根）
`{"cmd":"trim_place","targetId":"A2-06"}`	从修剪位搬回、放回原处 → 回待机位
`{"cmd":"estop"}`	程序级急停（之后所有命令会被拒绝，要**重启程序**才解除；真正断电请按物理急停）

反馈：`trim_pick` 先回 `retCode:1`（已开始），干完回 `retCode:0`（成功）；放回同理。如果收到 `retCode:2` + `a basket is already held; send trim_place first`，说明"上次搬运没结束"（处理见下👇）。

一直检测不到码怎么办？——识别位会等约 10 秒，失败直接回 `retCode:3`（`marker number N was not stable at the detection pose`），并且**机械臂会自动回到待机位**（与种子流程一致，2026-10-07 起）；排查原因（码被挡 / 光照 / 点位偏移）后，**重新发一条同样的 `trim_pick`** 即可重试，不需要做别的操作。

状态文件（小白也要懂的一条）：

- 运行程序把"正在搬的花篮"记在 `~/tl_fork_teach/active_transfer.yaml`；
- 正常 **trim_pick → trim_place 走完整圈后，这个文件会自动消失**；
- 如果搬运/回位**半路被 Ctrl+C 打断**，文件会残留——下次启动就会看到：

```
Active fork transfer state found: basket number=30, phase=hold1. …
Startup: a basket is still held (active fork state); the arm will NOT auto-move. …
```

意思 = "记账本里还有一单没完成"，程序出于安全**拒绝新任务**。

怎么处理（先肉眼看叉子上有没有花篮！）：

- 叉是空的（多数情况）→ 清掉记账本即可：

```
cat ~/tl_fork_teach/active_transfer.yaml      # 看一眼
rm ~/tl_fork_teach/active_transfer.yaml       # 删除（不影响示教数据）
```

- 真叉着花篮 → 别删！重启运行程序后发 `{"cmd":"trim_place","targetId":"<对应盆>"}`，让它走完返回自动放回。

11. 已经示教了哪些点（截至 2026-10-08，以第 11.3 节工具实际刷新为准）

11.1 种子（basket_aruco.yaml）——已完整示教 14 盆，全部是上排

每盆含：**检索位 + 落种接近位(A) + 落种释放位(P)**。

盆号 → 编号	盆号 → 编号
25 → A2-01	55 → A3-07
26 → A2-02	56 → A3-08
29 → A2-05	62 → A3-14
30 → A2-06	126 → B2-06
34 → A2-10	131 → B2-11
41 → A2-17	135 → B2-15
	151 → B3-07
	162 → B3-18

（合计 14 盆，全部 A2/A3/B2/B3 上排；下排种子盆还没教。）

⚠️ **A3 排码贴反（已于 2026-10-08 用软件修正，4 个脚本内置 `--reversed-rows A3`）：** A3 排的码是倒着贴的（本该 ID48→ID71，实际贴成 ID71→ID48；A3-01 格上贴的是 ID71）。上面 A3 三盆的"实际贴码 / 实际在第几格 / 修正后用什么命令"：

编号	实际贴的码	实际在第几格	这盆现在用哪条命令
55	ID54	第 18 格	**A3-18**
56	ID55	第 17 格	**A3-17**
62	ID61	第 11 格	**A3-11**

旧写法 `A3-07 / A3-08 / A3-14` 修正后会指向"没教过的格"（收到 `not taught`）。想把 **A3-08 / A3-14** 也教上：用新版示教程序照老流程教即可——现在输入 56 / 62 会正确匹配到第 8 / 14 格上的码（ID64 / ID58），不会再被弹到排后面去。**A3 排的教与跑都请用新版（`TL_lowerobs_test`）脚本**（老版 `TL` 没有这个修正）。

11.2 叉子（fork_basket_teach.yaml）——已示教 1 盆 + 3 个全局点

项目	内容	状态
30 号（A2-06）	检索位 + 叉接近位(F) + 水平插入位(I) + 竖直抬起位(L)	✅ 全齐，可跑
W 高位载运点（上排用）	示教值	✅ 已教

T 暂放位（上排用）	示教值	✅ 已教
W / T 高位载运点·暂放位（**下排专用**）	——	❌ 待教（教下排第一盆时按 W、T；教过之后 `list_taught.sh` 会显示"(下排专用)"两行）
O 待机位（全局）	已统一为**种球默认位** `[-23.246, -14.220, 53.738, 4.389, 26.746, -3.532]`（运行端 2026-10-07 起按新位执行；YAML 里还是旧值，下次按 O 会刷新）	✅
下排观测位 A / B（全局）	A 侧：需记录（v→拖→d→g）；**B 侧：不必记录**——运行端自动按 "A 侧 + J1 180°" 推导（2026-10-07 起），手动记录优先	⚠ A 侧尚未记录
修剪位 1 / 2（全局）	程序写死的固定值（第 13 章有数值）	✅ 已是新读数

11.3 怎么随时看最新、最准的清单（新增小工具，推荐每次用前跑一遍）

```
# 列出全部已示教盆 + 每个点的关节值（只读 YAML，不动机械臂）：
bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh

# 想看某一盆的完整坐标（含 TCP）：
bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh --basket A2-06 --full

# 想看 TCP 数值：
bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh --full
```

小提示：A3 排已启用贴反换算——工具里 A3 的位置列直接就是**可用命令名**（如 编号 55 显示为 A3-18），照它发命令即可。

输出示例（真实数据）：

```
===== 种子示教（basket_aruco.yaml） =====
  已示教 14 / 192 盆
  *  25  A2-01
       检索位   关节=[-83.1, 67.7, 3.9, -2.1, -5.4, -1.4]
       落种接近位(A)   关节=[…]
       落种释放位(P)   关节=[…]
…

===== 叉子示教（fork_basket_teach.yaml） =====
  已示教 1 / 192 盆
  *  30  A2-06
       检索位   关节=[…]
       叉接近位(F)   关节=[…]
       水平插入位(I)   关节=[…]
```

```
    竖直抬起位(L)  关节=[…]
    [全局] W 高位载运点  关节=[…]
    [全局] T 暂放位  关节=[…]
    [全局] W 高位载运点(下排专用)  关节=[…]   ← 教过下排 W 后出现
    [全局] T 暂放位(下排专用)  关节=[…]
    [全局] O 待机观测位  关节=[…]
```

两版共用一份数据，这个工具在哪个目录下跑都一样；老版目录里没有它，就从新版目录跑。

12. 现在就能运行的命令（复制即用）

前提（每次）：① 驱动已在跑（第 4 节）；② 只开一个程序；③ E-stop 在手边。

12.1 种子：下面 14 个盆都能跑（A3 排用新版，其余老版新版都行）

第一步，开一个终端启动种子运行：

```
# 新版（跑 A3 排必须用这个；其它盆行为与老版一致）：
bash ~/Desktop/TL_lowerobs_test/scripts/run_seed_mqtt.sh

# 老版（除 A3 排外的盆都可以）：
bash ~/Desktop/TL/scripts/run_seed_mqtt.sh
```

第二步，另开一个终端发命令（**一次发一条，等它跑完再发下一条**）：

```
# 25 → A2-01
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A2-01"}' -q 1
# 26 → A2-02
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A2-02"}' -q 1
# 29 → A2-05
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A2-05"}' -q 1
# 30 → A2-06
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A2-06"}' -q 1
# 34 → A2-10
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A2-10"}' -q 1
# 41 → A2-17
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A2-17"}' -q 1
# 55 → A3-18（A3 排贴反修正后；A3 排用新版跑）
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A3-18"}' -q 1
# 56 → A3-17（修正后）
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A3-17"}' -q 1
# 62 → A3-11（修正后）
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"A3-11"}' -q 1
# 126 → B2-06
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"B2-06"}' -q 1
# 131 → B2-11
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"B2-11"}' -q 1
# 135 → B2-15
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"B2-15"}' -q 1
# 151 → B3-07
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"B3-07"}' -q 1
# 162 → B3-18
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"B3-18"}' -q 1
```

12.2 叉子：30 号（A2-06）能跑全套（用新版）

```
# 第一步：启动叉子运行（新版）
bash ~/Desktop/TL_lowerobs_test/scripts/run_fork_mqtt.sh

# 第二步：另开终端，叉走（会停在修剪位 2 等剪根）
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"trim_pick","targetId":"A2-06"}' -q 1

# 第三步：剪根结束后，放回
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"trim_place","targetId":"A2-06"}' -q 1
```

12.3 急停（备着）

```
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"estop"}' -q 1
```

12.4 以后教了新盆，怎么自己拼命令

- 盆号 → 编号换算：看第 1 节表格（或第 11.3 节工具输出的 `盆号 编号`）；
- 套模板，把 `targetId` 换成你的编号：

```
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' -t
node/arm/01/down -m '{"cmd":"seed","targetId":"<编号，如 B3-07>"}' -q 1
```

13. 程序里最该认识的几个数字（关键参数在哪）

不用背，知道"出问题改哪里"就行。改完**重启对应程序**即生效（不用编译）。

数字 / 名字	在哪	意思
`--fixed-pick-base-mm 398.53 -193.69 211.63`	`scripts/run_seed_mqtt.sh`	固定抓种子点的位置（机械臂底座坐标，mm）
`--final-grasp-z-mm 218.880`	`scripts/run_seed_mqtt.sh`	抓取高度
`--observation-joints -23.246 …`	`scripts/run_seed_mqtt.sh`	种子流程的待机/观测位（关节角）
`--observation-joints -23.246 …`	`scripts/run_fork_mqtt.sh`	叉子流程的**待机位**（启动/作业结束回位；2026-10-07 起与种子统一；示教程序 O 键也去这里并记录）
`TRIM_HOLD_1 / TRIM_HOLD_2`	`fork_basket_cycle.py` 顶部（4 行 = 2 个点位×{关节,直角坐标}）	两个固定修剪点位（写死，不用教）；当前值： P1 关节 `-1.402, 2.199, -1.109, -18.450, -87.567, 21.143`；P1 直角 `372.383, -11.174, 512.929, 2.081, 1.527, 1.107` P2 关节 `-10.992, -51.448, -35.678, -12.270, -175.183, 6.713`；P2 直角 `371.473, -39.149, 175.217, -2.587, 1.529, -0.448`
`LOWER_ROW_INDEXES = (0,3,4,7)`	`fork_basket_cycle.py` 顶部	哪几排算"下排"（A1/A4/B1/B4）
起始位关节 `teach_start_joint_deg(_b)`	两个示教程序	A 侧起始位；B 侧 = A 侧 J1 加 180°
`active_transfer.yaml`	`~/tl_fork_teach/`	叉子"进行中作业"的记账文件（第 10 节）

14. 怎么把 MQTT 命令发出去（两种方式）

方式 A：终端命令（推荐）

模板（换掉 `cmd` 和 `targetId` 即可）：

```
mosquitto_pub -h 116.205.245.172 -p 1883 -u shapandevice -P 'qwer1234!!' \
  -t node/arm/01/down -m '{"cmd":"trim_pick","targetId":"A2-06"}' -q 1
```

账号是内部凭据，不要发给无关的人。没装 mosquitto 工具就用下面方式 B。

方式 B：图形工具 MQTTX（桌面有安装包）

- 新建连接：Host `116.205.245.172`，Port `1883`，Username `shapandevice`，Password 见脚本；连接；
- 订阅 `node/arm/01/up`（看反馈）；
- 向 `node/arm/01/down` 发送消息，内容就是上面 `-m` 后面的那段 JSON。

在哪能确认连上了：运行程序启动日志里有 `MQTT connected and subscribed: node/arm/01/down` 才算就绪。

15. 怎样下载/上传程序、怎样更新程序

15.1 把程序从电脑传到 Nano（上传）

方式 A：WinSCP 图形工具（推荐小白）

- 电脑上装 WinSCP（免费）；
- 新建连接：主机填 Nano 的 IP，用户名 `nvidia`，密码登录；
- 左边是电脑文件、右边是 Nano 文件，**直接拖文件夹**即可（比如把 `TL_lowerobs_test` 拖到右边 `~/Desktop/`）。

方式 B：scp 命令（Windows 自带）

在 Windows 的"终端 / PowerShell"里：

```
# 把整个文件夹从电脑传到 Nano 桌面：
scp -r "C:\Users\DragonAfterAll\Desktop\TL_lowerobs_test" nvidia@<Nano 的
IP>:"/home/nvidia/Desktop/"

# 只传一个文件（例如刚改好的脚本）：
scp "C:\Users\DragonAfterAll\Desktop\某文件" nvidia@<Nano 的
IP>:"/home/nvidia/Desktop/TL_lowerobs_test/scripts/"
```

方式 C：U 盘——拷进去，插到 Nano 上再拷到 `~/Desktop/`。

方式 D：从 Nano 下载回电脑（反向 scp）：

```
scp -r nvidia@<Nano 的 IP>:"/home/nvidia/Desktop/TL_lowerobs_test"
"C:\Users\DragonAfterAll\Desktop\"
```

15.2 传完之后必做两件事

```
# ① 补执行权限（Windows 文件本来就没有"可执行"属性）
chmod +x ~/Desktop/TL_lowerobs_test/scripts/*.sh

# ② 如果脚本报 "$'\r': command not found" 之类（Windows 记事本编辑过的痕迹），修一下换行符：
sed -i 's/\r$//' ~/Desktop/TL_lowerobs_test/scripts/*.sh
```

更省事的习惯：启动脚本一律用 `bash 脚本` 形式，就绕开了权限问题。

15.3 怎样更新程序（三种场景）

场景	怎么做	生效方式
改了一个/几个 .py 源码文件	传上去覆盖对应 `src/...` 里的文件	**重启对应脚本即生效，不用编译**（对照表见 18.3）
整个版本包更新（整文件夹替换）	传上去覆盖整个文件夹	重启脚本；**示教数据不在程序文件夹里**（在 `~/tl_fork_teach`、`~/tl_mock_pick_place`），不会

		丢
只改了脚本参数（如抓取高度）	编辑 `scripts/run_seed_mqtt.sh`	重启脚本生效

更新完怎么确认成功：启动程序看打印特征（第 5.3 节）。

16. 数据文件都存在哪（别乱删！）

文件	内容	能删吗
`~/tl_mock_pick_place/basket_aruco.yaml`	种子示教数据（每盆检索位、落种位、观测位）	❌ 删了要重教
`~/tl_fork_teach/fork_basket_teach.yaml`	叉子示教数据（每盆 F/I/L、全局 W/T/O、观测位）	❌ 删了要重教（每次写入会自动备份到同目录 `archive/`）
`~/tl_fork_teach/active_transfer.yaml`	叉子"进行中作业"记账（第 10 节）	✅ 确认叉上无花篮时可删
`~/tl_handeye_data/best/handeye_best.yaml`	相机手眼标定（相机的"眼睛度数"）	❌❌ 千万不要删/改

好习惯：教完一批盆后，把两个主 YAML 复制一份留底（例如 `cp 文件名 文件名.bak-日期`）。

17. 二次开发（一）：所有程序逐一详解

这一章把系统里**每个程序**都过一遍：它是干什么的、怎么跑、里面有哪些关键函数、想改东西动哪里。

先看 17.1 的地图，然后按需跳到对应小节。

17.1 程序地图（谁启动谁、谁继承谁）

```
启动脚本（scripts/）
├── teach_seed.sh   → tl_hand_deploy.aruco_basket_teach    （种子示教）
├── teach_fork.sh   → tl_fork_teach.fork_basket_teach      （叉子示教，也是叉子程序的“基类”）
├── run_seed_mqtt.sh→ tl_hand.mock_pick_place              （种子运行）
├── run_fork_mqtt.sh→ tl_fork_teach.fork_trim_mqtt         （叉子运行入口）
│                        └── 继承 fork_basket_cycle（搬运核心）
│                                 └── 继承 fork_basket_teach（相机/运动/示教基础）
└── list_taught.sh  → list_taught.py                       （查已示教点，V3 新增）

协议层
tl_hand/mqtt_protocol.py       （MQTT 命令解析/反馈编码/客户端；叉子程序也用它）
```

日常只碰 6 个文件：4 个启动脚本 + 两个示教程序（改按键）+ `fork_basket_cycle.py`（改修剪点位）。**其余 30 多个文件是标定工具、测试程序、历史版本，一般不用动**（17.10 有清单）。

17.2 种子示教程序 `tl_hand_deploy/aruco_basket_teach.py`

项目	内容
启动	`bash ~/Desktop/TL_lowerobs_test/scripts/teach_seed.sh`（老版脚本同文件名）
作用	拖动示教每盆的"检索位 / 落种接近位 / 落种释放位"，写入 `~/tl_mock_pick_place/basket_aruco.yaml`
运行流程	启动 → 去 A 侧起始位 → 循环：相机画面 + 按键
存文件特点	每个位置都以**两种形式**存：相对码的位姿（`T_marker_tool0`，用于换算）+ 绝对记录（`taught_joint`/`taught_tcp`，运行程序直接使用）；写文件时自动备份到 `archive/`

关键函数（在代码里 Ctrl+F 搜这些名字）：

函数	作用
`select_basket()`	B 键：锁定当前可见码 + 记检索位
`record_pose(name)`	A/P/F/I/L 键：把当前位姿记成 `basket_approach` / `basket_release` / `fork_*`

`record_transfer_pose()`	T 键：记全局暂放位 `fork_transfer_pose`
`_record_lower_observation()` / `_goto_lower_observation()`	d / g 键：记录 / 回下排观测位（新版专有）
`return_to_selected_search_pose()`	R 键：回检索位
`_write_output(data)`	写 YAML（含备份）
`run()`	主循环（相机 + 键盘）

17.3 种子运行程序 `tl_hand/mock_pick_place.py`

项目	内容
启动	`bash .../run_seed_mqtt.sh`
作用	等 MQTT `seed` 命令 → 固定点抓种子 → 识别盆 → 放种 → 回位 → 反馈 4 条
继承	`ReferenceOverheadTest`（公共的相机/运动基础，在 `overhead_test.py`）

关键函数：

函数	作用
`localize_basket()`	识别目标盆（含**下排路由**：下排盆先去观测位再看码）
`_lower_observation_joint(side)`	取观测位关节（命令行参数 > YAML 记录）
`_auto_seed_pick_place()`	一次完整的"抓种→放种"周期
`_handle_mqtt_command()` / `_process_mqtt_commands()`	MQTT 命令入口与循环处理
`_mqtt_publish_pending()` / `_mqtt_fail_pending()`	两阶段反馈（seed → seed_place）
`_hand_*` 系列	灵巧手控制：使能、位置读写、力度分级、level5 闭合检测（ModBus）
`execute_pick()` / `move_to_pick_target()`	抓种子动作
`return_observation()` / `smart_return()`	收尾回待机位

17.4 叉子示教程序 `tl_fork_teach/fork_basket_teach.py`

项目	内容
启动	`bash .../teach_fork.sh`
作用	拖动示教每盆 F/I/L + 全局 W/T/O + 下排观测位，写入 `~/tl_fork_teach/fork_basket_teach.yaml`；**它还是叉子运行程序的"基类"**（运行程序继承它拿相

	机/运动能力）
特点	带**动作校验**：插入/抬起不平不直会被拒绝（阈值见第 8 章）；内部有一个"阶段机"（idle→selected→lifted→…）防止乱按

关键函数：

函数	作用
`_select_marker()`	B 键：锁码 + 记检索位
`_record_pose(name)`	F/I/L 键：记录 + 校验（水平/竖直/姿态）
`_record_carry_waypoint()` / `_record_transfer()`	W / T 键：全局载运点 / 暂放位
`_record_observation()`	O 键：待机观测位
`_record_lower_observation()` / `_goto_lower_observation()` / `_switch_side()`	d / g / v 键（下排观测位三兄弟）
`_move_joint(target)`	核心运动：流式 ServoJ 到目标关节（所有自动运动都走它）
`_open_servoj()` / `_close_servoj()` / `_check_estop()`	伺服流开关 / 急停检查
`_return_to_search()`	R 键：回检索位

17.5 叉子搬运核心 `tl_fork_teach/fork_basket_cycle.py`

项目	内容
作用	真正"干活"的逻辑：`trim_pick` 的拿取搬运 + `trim_place` 的放回；被 `fork_trim_mqtt.py` 继承
顶部常量	`TRIM_HOLD_1/2`（修剪位 1/2，4 行）、`LOWER_ROW_INDEXES`（下排判定）——**改点位就改这里**（第 13 章）
与驱动交互	用 `/tl_driver/coord_transform` 服务做逆解（IK）、`/tl_driver/set_servoj_pos` 话题流式发关节、`open/close_servoj` 服务开关伺服

关键函数：

函数	作用
`_localize_marker(id)`	识别目标码（含下排观测位路由入口）
`_prepare_targets(...)`	把示教的 F/I/L 换算到"本次识别到的实际位置"
`_solve_ik(...)`	调驱动做逆解
`_stream_joint_path(...)`	关节空间流式运动（大动作都由它执行）
`_move_cartesian_line(...)`	直角空间直线运动（修剪位 1→2 的平移/下降）

`_interpolate_joint_path(...)`	直线走不通时的兜底插值
`_carry_transfer(...)` / `_carry_translate_segmented(...)`	载运段的安全搬运
`_pick_to_temporary()`	**trim_pick 全流程**：识别→F→I→L→W→T→修剪位 1→修剪位 2
`_return_to_original()`	**trim_place 全流程**：按记录逐段逆向返回→清状态→回待机位
`_write_state()` / `_load_state()` / `_clear_state()`	作业状态文件（第 10 节）读写清

17.6 叉子运行入口 `tl_fork_teach/fork_trim_mqtt.py`

项目	内容
启动	`bash .../run_fork_mqtt.sh`
作用	MQTT 命令分发：`trim_pick` / `trim_place` / `estop`；启动时读状态文件做保护（有残留状态就不自动动臂）

关键函数：`resolve_fork_basket()`（targetId→盆号，支持 `A2-06` 和纯数字）、`_handle_trim_pick()` / `_handle_trim_place()`（含 retCode 反馈）、`run()`（启动保护 + 命令循环）。

17.7 MQTT 协议模块 `tl_hand/mqtt_protocol.py`

（`tl_hand_deploy/mqtt_protocol.py` 是旧副本）

项目	内容
常量	`SUPPORTED_COMMANDS`（支持的命令名）、`RET_SUCCESS=0 / RET_STARTED=1 / RET_OCCUPIED=2 / RET_RECOG_FAILED=3`
函数	`target_id_to_basket_number()`（"A2-06"→30）、`parse_command()`（解析 JSON 命令）、`encode_feedback()`（拼反馈）
类	`MqttClient`：连接 broker、订阅命令话题、收命令入队（`drain_commands()`）、发反馈（`publish_feedback()`）
想加新命令	改两处：① 这里 `SUPPORTED_COMMANDS` 加名字；② 对应运行程序加处理函数

17.8 启动脚本（4 个 .sh）一文看懂

每个脚本都是同一个套路（以 `run_fork_mqtt.sh` 为例）：

```
source /opt/ros/humble/setup.bash          # ① 加载 ROS 环境
source ~/Desktop/tl_robot_ros2_py-arm/install/setup.bash   # ② 加载驱动消息包
export PYTHONPATH="$root_dir/src/tl_fork_teach:…"          # ③ 把源码目录放进搜索路径
python3 -m tl_fork_teach.fork_trim_mqtt --各种参数…        # ④ 启动程序
```

- 想改参数（抓取点、速度、MQTT 地址等）：直接编辑脚本里的 `--参数`，重启生效；
- `"$@"` 表示"你在命令行加的额外参数也会原样传给程序"，可以临时覆盖，例如：

  `bash run_fork_mqtt.sh --trim-hold-speed-deg-s 8`（把修剪段速度调慢）。

17.9 查点小工具 `list_taught.py`（V3 新增）

只读两份 YAML、不连机械臂。用法见第 11.3 节。以后教完新盆想"列出清单/复制坐标"就用它。

17.10 其它程序一览（标定 / 测试 / 历史，一般不用碰）

文件（在 tl_hand_deploy 或 tl_hand 包里）	一句话
`handeye_calibrate.py` / `handeye_collect_node.py`	相机手眼标定：采集数据 + 求解标定（产出 `~/tl_handeye_data/best/handeye_best.yaml`）
`handeye_accuracy.py` / `handeye_validate_report.py`	标定精度统计 / 验证报告
`convert_handeye_frame.py`	把"法兰坐标系"的标定换算到实际安装的 TCP 坐标系
`roh_hand_node.py` / `roh_protocol.py` / `roh_hand_test.py`	灵巧手（ROH-LiteS001）：控制节点 / 通信协议 / 独立测试
`gripper_do_test.py`	测试数字输出→手爪开合接线
`system_check.py`	系统自检（驱动 / 相机 / 灵巧手是否就绪）
`safe_arm_node.py` / `safe_move_cli.py`	早期"安全代理"方案（命令先过一层代理再动）；现在的流程不用
`mock_place_teach.py`	旧版"落种位置"记录工具（已被新的示教程序取代）
`mock_pick_place_headless.py`	无窗口版种子运行（没有屏幕时可用）
`mock_pick_place_movej.py` / `movej_click.py` / `overhead_test.py` / `overhead_movel_test.py`	调试/实验程序（MoveJ 版本、鼠标到位、早期测试）
`dynamic_grasp_test.py`	"动态高度抓取"实验（抓取高度由识别到的表面 Z 决定）
`validation.py` / `geometry.py` / `reference_compat.py` / `basket_aruco.py`	工具/兼容模块（校验函数、几何换算、编号工具），不直接运行

`fork_basket_controller.py`	最早的"单文件版"叉子控制器（已被 `fork_basket_teach/cycle/trim_mqtt` 三件套取代）
`tl_hand_deploy/mock_pick_place.py` 等一批同名文件	**旧版副本**：稳定版（老版）的种子程序引用的是这一份；新版验证包引用的是 `tl_hand/` 里的那一份

18. 二次开发（二）：改码流程与调试

18.1 先记住三条

- **只在"新版验证包"（`~/Desktop/TL_lowerobs_test`）里改**——`~/Desktop/TL` 是稳定版，别动；
- 改完**不用编译**（本项目的程序都是 `PYTHONPATH` 直读 `src` 源码），传到 Nano 后重启对应脚本就生效；只有改"驱动包"才需要 `colcon build`（对照表见 18.3）；
- 改之前先备份（或记下改了什么），一次只改一处。

18.2 "我想改 X → 改哪个文件"

想做的事	改哪里
修剪位 1 / 2 的数值	`fork_basket_cycle.py` 顶部 `TRIM_HOLD_…`（**关节、直角坐标成对改**）
抓种固定点 / 抓取高度 / 待机位	`scripts/run_seed_mqtt.sh` 里的参数
"下排"的定义（哪几排算下排）	`fork_basket_cycle.py` 的 `LOWER_ROW_INDEXES`；种子侧在 `mock_pick_place.py` 的 `(0, 3, 4, 7)`
新增一条 MQTT 命令	`mqtt_protocol.py` 的 `SUPPORTED_COMMANDS` + 运行程序里加处理函数
示教程序加一个新按键	对应示教 `.py` 的按键处理段（搜索 `ord("d")` 附近的代码）
识别码大小 / 相机参数	脚本里的 `--marker-length-m` 等参数
运动速度	脚本里的 `--servoj-speed-deg-s`（示教）/ `--trim-hold-speed-deg-s`（修剪段）

18.3 改文件 ↔ 哪个脚本生效 ↔ 要不要编译（总表）

先记结论：本项目（`TL` 和 `TL_lowerobs_test` 里你日常用的所有程序）改任何文件都【不需要编译】。 原因：4 个主脚本启动时把 `src` 目录排在最前面塞进 `PYTHONPATH`，再用 `python3 -m` 直接加载 `.py` 源码——Python 每次运行都重新读取源码文件，所以改完只要"重启对应脚本"就生效。（`.pyc` 缓存会自动刷新，不用管。）

你改的文件	由哪个脚本加载 / 生效于	要编译吗	怎么生效
`src/tl_hand_deploy/tl_hand_deploy/aruco_basket_`	`scripts/teach_seed.s`	❌	重启

`teach.py`（种子示教）	`h`	不用	`teach_seed.sh`
`src/tl_hand/tl_hand/mock_pick_place.py`（种子运行）	`scripts/run_seed_mqtt.sh`	❌ 不用	重启 `run_seed_mqtt.sh`
`src/tl_fork_teach/tl_fork_teach/fork_basket_teach.py`（叉子示教，也是**叉子运行的基类**）	`scripts/teach_fork.sh`，**同时影响** `scripts/run_fork_mqtt.sh`	❌ 不用	用到哪个重启哪个（只改了示教相关就只重启示教）
`src/tl_fork_teach/tl_fork_teach/fork_basket_cycle.py`（搬运核心 / 修剪位常量）	`scripts/run_fork_mqtt.sh`（经 `fork_trim_mqtt` 继承）	❌ 不用	重启 `run_fork_mqtt.sh`
`src/tl_fork_teach/tl_fork_teach/fork_trim_mqtt.py`（叉子运行入口）	`scripts/run_fork_mqtt.sh`	❌ 不用	重启 `run_fork_mqtt.sh`
`src/tl_hand/tl_hand/mqtt_protocol.py`（MQTT 协议）	**两个运行程序都用到：** `run_seed_mqtt.sh`、`run_fork_mqtt.sh`	❌ 不用	用到哪个重启哪个
`scripts/*.sh`（改参数、改路径）	它自己	❌ 不用	重新执行该脚本
`scripts/list_taught.py`（查点工具）	`scripts/list_taught.sh`	❌ 不用	直接再跑一次
数据 YAML（示教数据 / 作业状态文件）	运行程序启动时读取	❌ 不用	重启运行程序（状态文件另见第 10 节）
驱动包 `~/Desktop/tl_robot_ros2_py-arm/src/...`（`tl_driver` 节点、消息/服务定义等）	驱动：`ros2 launch tl_driver …`	✅ **要**	见下表下方

唯一需要编译的情况——改了驱动包（`tl_driver` 节点、消息/服务定义 `.msg/.srv` 等）：

```
cd ~/Desktop/tl_robot_ros2_py-arm
colcon build                                  # 全编最稳；或只编改动的：colcon build --packages-select tl_driver
source install/setup.bash
# 然后：重启驱动终端，再重启用到它的程序（示教/运行程序）
```

判断口诀：

- 改的东西在 `~/Desktop/TL_lowerobs_test` 或 `~/Desktop/TL` 里 → **不用编译**，重启脚本；
- 改的东西在 `~/Desktop/tl_robot_ros2_py-arm`（驱动包）里 → **要编译**（`colcon build`）；
- 改的是"消息 / 服务定义"（`.msg/.srv`，一般在驱动包里）→ **一定要编译**，而且编译后所有用到它的程序都要重启。

小众例外：老版目录里个别旧脚本（早年那批用 `ros2 launch / ros2 run` 启动"安装包"的，比如 `run_fixed_pick_mqtt.sh` 一类）走的是编译产物，改源码后要 `colcon build` 才生效。你现在日常用的 4 个主脚本都不是这种方式——**拿不准时看一眼脚本里有没有 `python3 -m`：有，就是直读源码，不用编译：**

```
grep -n "python3 -m\|PYTHONPATH" scripts/*.sh
```

18.4 标准开发流程（5 步）

```
① 在电脑上改代码（推荐 VS Code 打开 TL_lowerobs_test 文件夹）
② 传到 Nano 对应位置（第 15 节）
③ 在 Nano 上做语法检查：  python3 -m py_compile 你改的文件.py
   （没输出=语法没问题；报错会指出第几行，先修语法）
④ 空载测试：不放手爪/花篮，跑一遍示教程序或发一条命令，看打印
⑤ 测试稳定后，才考虑合并回 TL 稳定版
```

18.5 调试常用（在 Nano 上）

```
ros2 topic list                          # 看系统里有哪些话题
ros2 topic echo /joint_states --once     # 看一次关节状态（臂是否在反馈）
ros2 topic echo /tcp_pose --once         # 看一次末端位置
```

- 程序报错就看终端**最后几行**（Python 报错最后一行是原因）；
- 直接修改（不用电脑）也可以用 nano 编辑器：`nano 文件路径`，Ctrl+O 保存、Ctrl+X 退出。

18.6 强烈推荐：VS Code 远程开发

- VS Code 装插件 **Remote - SSH**；
- 左下角绿色按钮 → Connect to Host → 输入 `nvidia@<Nano 的 IP>`；
- 之后直接打开 Nano 上的文件夹（如 `/home/nvidia/Desktop/TL_lowerobs_test`），**像编辑本地文件一样改代码，保存就直接保存到 Nano**，不用来回传。

18.7 红线

- 别在 `~/Desktop/TL`（稳定版）里直接改；
- 改完必须先**空载测试**再上真种子/花篮；
- 数据文件（第 16 节）别乱动；修剪点位这种"关节+直角"必须成对修改。

19. 小白常见问题速查表

现象 / 提示	什么意思	怎么办
`Permission denied`	脚本没有执行权限（从 Windows 拷过去常见）	`chmod +x …/scripts/*.sh`，或始终用 `bash 脚本` 方式启动
按 B 报 `not visible / not enough stable samples`	手没停稳或码看不清	松开手、停稳 1~2 秒，再按一次 B
按 d 报 `/tcp_pose is stale`	同上，臂没停稳	等 1~2 秒再按 d
`a basket is already held` / `Active fork transfer state found`	叉子有"未完成搬运"的记账	见第 10 节：空的就 `rm` 记账文件；真叉着花篮就发 `trim_place`
B 区下排（B1/B4）叉取会去哪个观测位？	程序**自动按盆号**选侧（97–192 号盆 → B 侧）；B 侧**没有单独记录时自动 = A 侧记录把 J1 加 180°**（2026-10-07 起；场地 A/B 精确对称，实测两侧起始位 J1 正好差 180.000°）	看到日志 `derived from the A-side record rotated 180 deg` 就是走了自动推导；你手动记录过 B 则以记录为准
`lower-row observation pose for side A is not recorded`（retCode 3）	**A 侧**下排观测位还没记录（B 侧自 2026-10-07 起会自动推导，不再报错）	到示教程序：按 v 切到 A 侧 → 拖到能看清 A1/A4 码处 → 停稳 → 按 d → 按 g 验证；核实用 `list_taught.sh` 看 [全局] 行
`marker number N was not stable at the detection pose`（retCode 3）	叉取时在识别位等了约 10 秒都没能稳定看到码（被挡 / 光照 / 点位偏移；`got 0 samples` = 一帧都没看到）	程序会**自动回待机位**（2026-10-07 起，对齐种子流程）；排查后**重新发一条同样的 `trim_pick`** 重试
`basket number X is not taught`（retCode 3）	这个盆没做过**叉子示教**（注意：种子教过 ≠ 叉子教过，两套流程各自教各自的）	到叉子示教程序把该盆教一遍（B→F→I→L→R）；或先换一个已教过的盆测试
"默认观测位（−23.246…）"和"下排观测位"是一回事吗？	**不是**：默认观测位=待命站位（启动/收工用，**不看码**）；下排观测位=专门看下排盆码的识别位置（v→拖→d 记录）；上排盆则连观测位都不去（**直接检索位**）	三者数值、用途都不同；`seed A1-01`/`trim_pick A1-01` 去"下排观测位 A"，`seed A2-01` 去"A2-01 检索位"，谁都不去默认观测位（它只在出发前/收工后出现）
下次示教怎么从待机位回到下排观测位？	按 `g`——臂**自动**去"当前侧"已记录的观测位（不用手拖）；想先回待机位按 `0`	观测位只记一次、长期有效；要改位置就拖到位再按 `d` 覆盖。B 侧没单独记录时 `g` 会提示未记录——直接按 `v` 切侧后手拖即可（运行端会自动推导 B 侧）
下排盆要不要教 W / T？	**要教一对下排专用的**（2026-10-08 起）：教下排第一盆时按 W、T 就会自动存成"下排专用"	还没教下排那对之前：运行下排盆会沿用上排那对并打印 `WARNING: lower-row … not`

	（上排那对不受影响）；之后运行下排盆自动用下排那对	`taught`；下排 L→W 段无沿途碰撞检测，首次仍建议"空盆 + 降速"（`--servoj-speed-deg-s 10`）试跑确认
输入 A3-08，机械臂却跑到 A3 排**后面**去了（不在前面）	A3 排的 ArUco 码**贴反了**（本该 ID48→ID71，实际贴成 ID71→ID48）；程序按码定位，于是去了码所在的后排位置	已修正：4 个脚本内置 `--reversed-rows A3`（输入位置号自动换成该格实际贴的码）。已教 A3 盆的新命令见第 11.1 节（**A3-18 / A3-17 / A3-11**）；补教 A3-08 / A3-14 直接用新版教（输入 56/62 就对了）。**A3 的教与跑都用新版**，老版没有修正
按 `0` 为什么去的是"抓种子默认位"，不是去看花篮？	设计如此：待机位（`0`）自 2026-10-07 起统一为种球默认位 `[-23.246, …]`，只管启动/收工回位，**不看码**	看花篮用别的键：去 **A/B 起始位**按 `v`（`-83.668… / 96.332…`，即老观测位）；看**下排盆**按 `g`（先 `d` 记录）；**上排盆**直接去该盆检索位（输编号 + `B` 选中后按 `R`）
ArUco 码 ID 和盆编号怎么换算？	**编号 = ID + 1**（ID 0..191 ↔ 编号 1..192）	例：**ID34 → 编号 35 → A2-11**（减 1 除以 24 得排、余数+1 得第几格）
`Command rejected/stopped: … rcl node's context is invalid`	你在动作**没走完**时按了 Ctrl+C，程序被关了	重启程序重做即可；下次等动作走完再退出
`Returning with N direct ServoJ points.`	**正常**：一次自动运动开始了（约 N÷100 秒）	等它出现完成打印 `Returned: max joint error=… deg`
`accepted=False, convention=intrinsic_xyz`	标定文件的"验收标记"（正常启动信息）	不用管：脚本带了 `--allow-rejected`，会继续用
一堆 SciPy / NumPy `UserWarning`	环境版本提示	无视，不影响
识别不到码（`retCode 3`）	相机看不清盆上的码	检查码是否贴平、是否被遮挡、距离/光线是否合适；换个角度重试
机械臂完全不动	驱动没起 / 程序没在等命令 / 起了两个程序	先看驱动终端；再确认运行程序打印了 `MQTT connected`；检查没有开第二个程序
MQTT 连不上（运行程序卡住不发反馈）	网络或账号问题	检查 Nano 能上网；账号密码是否正确；重启运行程序
`bash: $'\r': command not found`	脚本被 Windows 编辑器改过换行符	`sed -i 's/\r$//' 脚本.sh`
示教窗口按键没反应	可能是相机帧或状态短暂卡顿	等几秒；持续卡死记录日志反馈

20. 最快上手路线（照这个顺序做就行）

```
第 ① 步：用 NoMachine 或 SSH 连上 Nano（第 3 节）
第 ② 步：终端 A 起驱动 → 终端 B 给脚本加权限 → 确认驱动在持续刷状态
第 ③ 步：跑一次查点工具：bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh
         先看看现在有哪些盆能干活（对照第 11 章清单）
第 ④ 步：拿现成命令练手（第 12 章）：用老版跑一条 A2-01 的 seed；用新版跑 A2-06 的
         trim_pick + trim_place，看机械臂走完整流程
第 ⑤ 步：学示教：种子示教教 1~2 个上排盆（编号→B→A→P→R）；
         叉子示教（新版：O → B→F→I→L→W→T→R）
第 ⑥ 步：要干下排盆（A1/A4/B1/B4）：换新版，先记录 A/B 两侧观测位（v→拖→d→g）
第 ⑦ 步（进阶）：想改程序 → 看第 17、18 章（每个程序干什么、改哪里、怎么测）
```

快捷索引（打印出来贴在手边）：

```
远程进 Nano：   NoMachine（图形）或 ssh nvidia@<Nano 的 IP>（命令行）
起驱动：        ros2 launch tl_driver tl_tcb605_driver.launch.py

老版（上排种子推荐）：~/Desktop/TL/scripts/
  teach_seed.sh | teach_fork.sh | run_seed_mqtt.sh | run_fork_mqtt.sh
新版（下排种子/叉子推荐）：~/Desktop/TL_lowerobs_test/scripts/
  teach_seed.sh | teach_fork.sh | run_seed_mqtt.sh | run_fork_mqtt.sh | list_taught.sh

查已示教点：    bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh
发命令（结构）：{"cmd":"seed / trim_pick / trim_place / estop","targetId":"A2-06"}
看反馈：        订阅 node/arm/01/up；retCode 0=成功 1=开始 2=被占 3=失败
动作完成标志：  "Returned: max joint error=…" / "Cycle complete …"
清叉子记账：    rm ~/tl_fork_teach/active_transfer.yaml（先确认叉上无花篮）
传程序：        WinSCP / scp / U 盘 → 传完 chmod +x
改代码：        只改 TL_lowerobs_test；传上去重启脚本生效（不用编译）
要不要编译：    改自己的程序（TL / TL_lowerobs_test 里）一律不用；只有改驱动包才 colcon build
纪律：          E-stop 在手边；只开一个程序；动作没走完别按 Ctrl+C
```